



Jump Consistent Hashing in Java



About me

- Java developer



The Routing Problem ($\% N$)

$\% N$ breaks stability when N changes

Mass key migrations

Cold caches, latency spikes

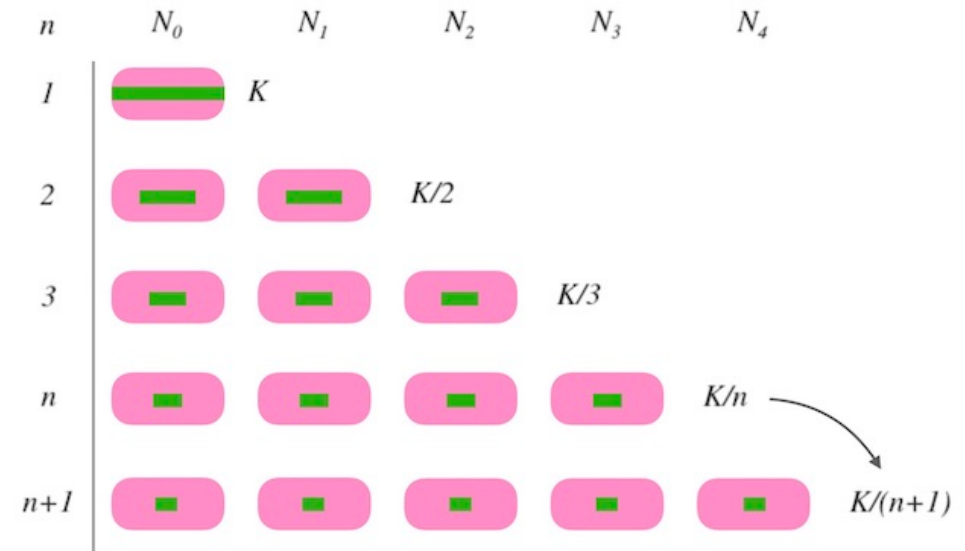


Why Jump Consistent Hash (JCH)

Simple, deterministic routing

$\approx 1/(N+1)$ keys move on add

Keys are distributed nearly evenly among buckets



How JCH Works

Hash each key into a stable 64-bit number

The hash drives a sequence of “jumps” over bucket indices

The bucket is the last index you can jump to while remaining $< N$

When N increases, only keys whose next jump is the new bucket move there

```
int ch(int key, int num_buckets) {  
    random.seed(key);  
    int b = 1; // bucket number before the previous jump  
    int j = 0; // bucket number before the current jump  
    while (j < num_buckets) {  
        b = j;  
        r = random.next();  
        j = floor((b + 1) / r);  
    }  
    return b;  
}
```



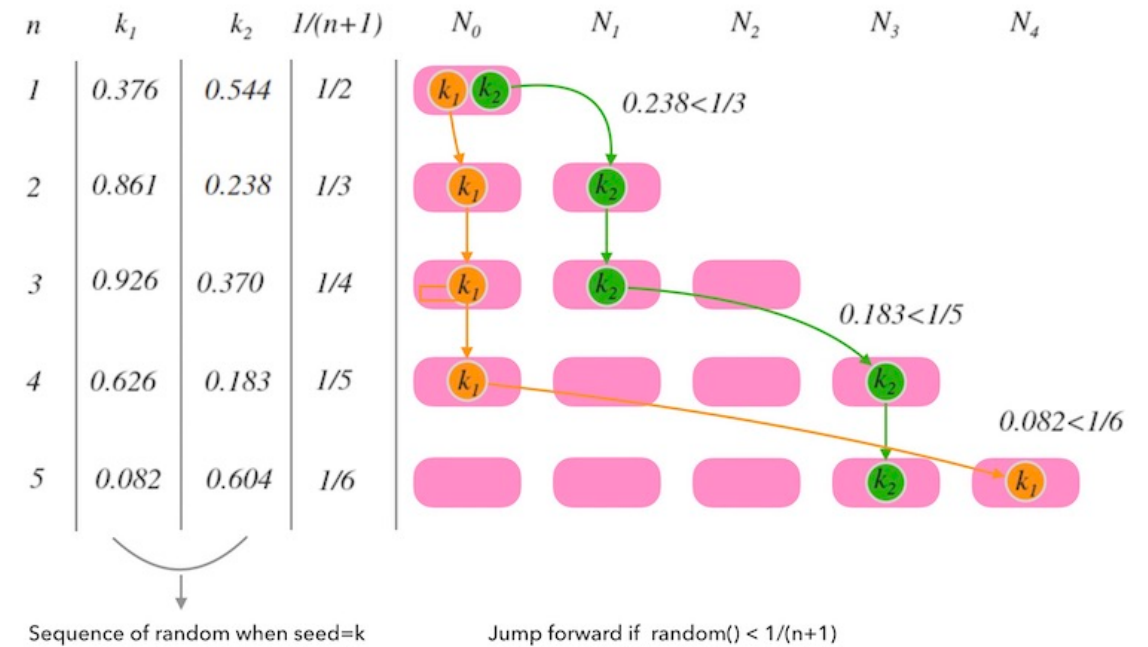
Adding a Bucket ($N \rightarrow N+1$)

Add 1 bucket

Migrating keys: $\approx 1/(N+1)$ with “threshold $N+1$ ”

These keys go to new bucket N

Time complexity $O(\log N)$, memory $O(1)$



Removing a Bucket ($N \rightarrow N-1$)

Around $1/N$ of keys move only if their bucket is removed

Keep bucket IDs contiguous $0..N-1$ using an indirection map

If removing a non-last bucket: swap with last $\rightarrow$ shrink N

All other keys stay put



JCH vs Ring Consistent Hash

Feature	Jump Consistent Hash	Ring Consistent Hash
Data Structure	None	Ring with virtual nodes (vnodes)
Key Movement	Minimal ($\approx 1/(N \pm 1)$)	Depends on vnode count
Lookup Speed	Expected $\sim O(\log N)$	$O(\log V)$ (binary search by V)
Overhead	Very low (no ring/vnodes)	Higher (manage many vnodes)
Ease of Use	Simple	More complex



Safe Resizing Practices

Add buckets one at a time to limit churn

Expect $\approx 1/(N+1)$ key migration per added bucket

On removal, swap+shrink with an indirection map to keep IDs contiguous

Monitor: % keys migrated, cache hit rate, latency

Throttle resize rate to avoid overload



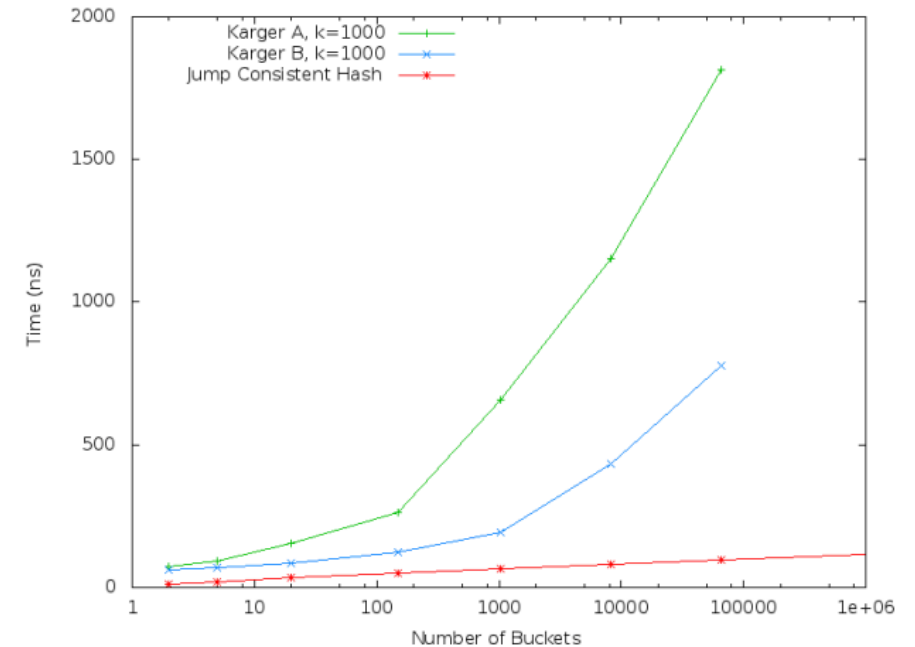
Performance and Scalability

Predictable latency as N grows (expected $\sim O(\log N)$)

Minimal key shuffling lowers load on storage/caches

Higher cache hit rates, smoother scale-out

Memory overhead $O(1)$



Java Implementation Tips

Use a stable 64-bit hash (e.g., Murmur3_128
→ `.asLong()`, `xxHash64`)

Avoid `hashCode()` (32-bit, JVM/impl-dependent)

Guava: `int bucket = Hashing.consistentHash(hash64, N);`

Pipeline: `key` → stable hash64 → JCH → bucket

Typical uses: Kafka partitioning, distributed caches,
worker routing



When to Use JCH

Simple, fast, $O(1)$ memory, minimal movement on resize, near-even distribution

Use when: Many buckets, frequent resizes; buckets $\approx$ equal capacity

No native weights; not for exact per-bucket load guarantees

Avoid/Alternatives: need weights/precise load $\rightarrow$ HRW/Rendezvous (weighted), Maglev (weighted), or Ring CH with tuned vnodes



Thank you

- Author: Denys Kuchmei
- My LinkedIn: <https://www.linkedin.com/in/denys-kuchmei-7a8259208/>
- Date: November 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)

